Mini Project Report on

Smart Personal Finance Management Using LLM-Based Decision Support

Submitted by

B Varshith (23BDS011)

Hari Hara Sudhan V S (23BEC021)

M Jagadesh (23BDS033)

J Ganesh (23BDS024)

Under the guidance of

Dr. Anushree D. Kini

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DHARWAD

27/03/2026

Certificate

This is to certify that the project, entitled **Smart Personal Finance Management Using LLM-Based Decision Support**, is a bonafide record of the Mini Project coursework presented by the students whose names are given below during 2025-2026 in partial fulfilment of the requirements of the degree of Bachelor of Technology in Computer Science and Engineering.

Roll No	Names of Students
23BDS011	B Varshith
23BEC021	Hari Hara Sudhan V S
23BDS033	M Jagadesh
23BDS024	J Ganesh

Dr. Anushree D. Kini
(Project Supervisor)

Contents

List of Figures v

List of Tables vi

1 Introduction 7

1.1 Background and Motivation 7

1.2 Problem Statement 8

1.3 Objectives 9

1.4 Scope of the Project 9

1.4.1 Frontend Scope 10

1.4.2 Backend Scope 10

1.4.3 AI Core Scope 11

1.5 Key Contributions 11

1.6 Organization of the Report 12

2 Related Work 13

2.1 Personal Finance Management Systems 13

2.1.1 Transaction Tracking Systems 13

2.1.2 Budget Visualization Tools 13

2.1.3 Investment Tracking Platforms 13

2.2 Large Language Models in Finance 14

2.2.1 Domain-Specific Financial Models 14

2.2.2 General-Purpose LLMs in Finance 14

2.2.3 Limitations of Current LLM Approaches 14

2.3 Multi-Agent Systems for Decision Support 15

2.3.1 Agent Orchestration Frameworks 16

2.3.2 ReAct and Chain-of-Thought Approaches 16

2.3.3 Application to Financial Reasoning 16

2.4 Workflow Automation in Finance 16
2.4.1 Rule-Based Automation 16
2.4.2 AI-Assisted Automation 17
2.5 Comparison with Existing Approaches 17
2.6 Gap Analysis 17
3 Data and Methods 19
3.1 System Architecture Overview 19
3.2 Directory Structure 20
3.3 Frontend Architecture 21
3.3.1 Technology Stack 21
3.3.2 State Management Strategy 22
3.3.3 Routing Architecture 23
3.4 Backend Architecture 24
3.4.1 Middleware Stack 24
3.4.2 Service Layer Design ..... **Error! Bookmark not defined.**
3.5 AI Core Architecture 25
3.5.1 Agent System 25
3.5.2 Query Routing System 28
3.5.3 Provider Failover 28
3.5.4 Circuit Breaker Pattern 29
3.6 Data Model Design 29
3.6.1 Core Design Principles 29
3.6.2 Model Categories 30
3.6.3 Transaction Model Detail ..... **Error! Bookmark not defined.**
3.7 Security Architecture 31
3.7.1 Authentication Mechanisms 32

3.7.2 Security Controls 33
3.7.3 Audit Event Types..... **Error! Bookmark not defined.**
3.8 Workflow Automation System ..... **Error! Bookmark not defined.**
3.8.1 Trigger Types..... **Error! Bookmark not defined.**
3.8.2 Action Types 33
3.8.3 Autopilot Lifecycle..... 34
3.9 Plugin System Architecture 34
3.9.1 System Components..... 34
3.9.2 Permission Scopes..... 35
3.10 Observability Stack 35
3.11 API Design 36
3.11.1 Versioning Strategy..... 36
3.11.2 Error Response Format..... 36
4 Results and Discussions 36
4.1 System Implementation 36
4.1.1 Implementation Statistics..... 37
4.1.2 User Interface Implementation 38
4.2 Multi-Agent Response Analysis 39
4.2.1 Response Metadata Structure..... **Error! Bookmark not defined.**
4.2.2 Conversation Insights Example 40
4.3 API Coverage Analysis..... 40
4.4 Security Implementation Results..... 41
4.4.1 Two-Factor Authentication 41
4.4.2 Account Lockout Implementation ..... **Error! Bookmark not defined.**
4.4.3 HTTP Security Headers 42
4.5 File Processing Capabilities..... 42

4.6 Performance Characteristics 43
4.6.1 Caching Strategy 43
4.6.2 Concurrency Management 44
4.6.3 Code Splitting ..... **Error! Bookmark not defined.**
4.7 Workflow Execution Results ..... **Error! Bookmark not defined.**
4.7.1 Cron Workflow Execution ..... **Error! Bookmark not defined.**
4.7.2 Event-Driven Execution ..... **Error! Bookmark not defined.**
4.8 Discussion of Results 44
4.8.1 Strengths of the Approach 44
4.8.2 Comparison with Initial Objectives 46
4.8.3 Limitations 46
4.8.4 Comparison with Related Work 47
5 Conclusion 48
5.1 Summary of Achievements 48
5.1.1 Technical Achievements 48
5.1.2 Product Achievements 49
5.2 Key Contributions 49
5.3 Lessons Learned 50
5.4 Future Work 50
5.4.1 Near-Term Enhancements 51
5.4.2 Medium-Term Enhancements 51
5.4.3 Long-Term Vision 51
5.5 Closing Remarks 52
References 53

List of Figures

1 Evolution from Traditional Finance Apps to LLM-Based Decision Support . . . 2
2 Three-Pillar System Architecture Scope 5
3 Evolution of Personal Finance Management Systems 9
4 Single-Agent vs Multi-Agent Architecture Comparison 10
5 Gap Analysis Summary and Solution Coverage 13
6 High-Level System Architecture Diagram 15
7 Hybrid State Management Architecture 18
8 Request Flow Through Middleware Pipeline 21
9 Multi-Agent LangGraph Workflow Architecture 24
10 LLM Provider Failover Chain with Circuit Breaker 25
11 Database Model Organization (48 Models across 9 Categories) 28
12 Defense-in-Depth Security Architecture 29
13 Autopilot Lifecycle: Safe Automation with Human Oversight 33
15 Implementation Statistics Visualization 37
16 Dashboard UI Layout with Chat-First Design 38
17 Agent Workflow Execution Trace Visualization 39
18 API Endpoint Distribution by Category 42
19 Two-Factor Authentication Flow with Account Lockout 43
20 File Processing Pipeline with Type Detection and AI Analysis 45
21 Workflow Execution Engine with Trigger Types and Actions 48
22 Feature Capability Comparison Across Systems 51
23 Project Achievement Summary 53
24 Future Development Roadmap 56

List of Tables

1 Comparison with Existing Personal Finance Systems 12
2 System Subsystems Overview 14
3 Frontend Technology Stack 17
4 Route Categories 19
5 Middleware Stack (Application Order) 20
6 Service Categories 22
7 Specialist Agents 23
8 Circuit Breaker Configuration 26
9 Database Model Categories 27
10 Authentication Methods 30
11 Security Control Matrix 31
12 Workflow Trigger Types 32
13 Workflow Action Types 33
14 Plugin System Components 34
15 Plugin Permission Scopes 34
16 Observability Stack 35
17 Implementation Statistics 36
18 API Endpoint Coverage by Category 41
19 Implemented Security Headers 44
20 Supported File Types and Processing 45
21 Objective Achievement Analysis 50

1 Introduction

1.1 Background and Motivation

Personal finance management has undergone significant transformation in the digital age. Traditional applications have focused primarily on passive record-keeping, displaying transactions, balances, and charts without providing actionable guidance. Most existing applications fall into one of two categories: they are either record-keeping tools that show balances and charts but do not help users decide what to do next, or they add a generic AI chatbot that talks about finance but does not reason through the user's full financial state in a structured way [2].

The emergence of Large Language Models (LLMs) has created new possibilities for intelligent financial assistance. However, simply adding a generic chatbot to a finance application does not address the fundamental limitation of passive data presentation. Users need systems that can analyze their complete financial picture, consider multiple factors simultaneously, and provide structured recommendations based on their unique circumstances.

This project, **Smart Personal Finance Management Using LLM-Based Decision Support**, addresses this gap by combining traditional finance tooling with structured AI reasoning and operational features around trust, automation, and collaboration. The system is designed to help users move from raw financial data to guided action through dashboards, AI chat, workflows, collaboration features, and automations.

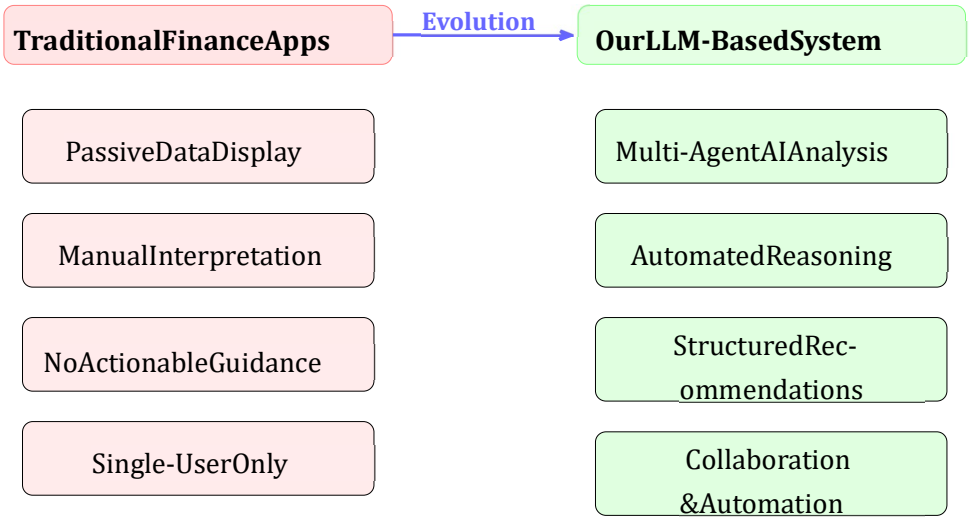


Figure 1. Evolution from Traditional Finance Apps to LLM-Based Decision Support

The motivation behind this project stems from several key observations:

1. **Passive Data Problem**: Existing finance applications excel at data collection but fail to convert that data into actionable insights.

2. **Single-Agent Limitations**: Generic AI chatbots lack the specialized knowledge required for comprehensive financial analysis.

3. **Trust Deficit**: Users are hesitant to act on AI recommendations when the reasoning process is opaque.

4. **Collaboration Gap**: Real financial planning often involves multiple stakeholders, yet most tools are designed for individual use.

5. **Automation Absence**: Manual execution of financial tasks leads to inconsistency and missed opportunities.

1.2 Problem Statement

Many personal finance applications stop at recording data. They do not reliably answer higher-value questions such as:

- Where is money leaking month to month?
- Should the user prioritize debt repayment or investing?
- Can a future goal be funded without breaking the current budget?
- How should shared financial work happen inside a family or small team?
- What are the trade-offs between different financial strategies?
- How can recurring financial tasks be automated safely?

The core challenge is to develop an intelligent personal finance platform that combines secure data handling, organization-aware collaboration, workflow automation, and specialist AI analysis to provide actionable financial guidance rather than passive data display.

1.3 Objectives

The primary objectives of this project are:

1. To develop a full-stack personal finance workspace with multi-agent AI reasoning capabilities that can analyze complex financial scenarios
2. To implement a chat-first finance assistant that routes prompts through specialist agents and returns synthesized responses with plans and workflow traces
3. To enable comprehensive file upload, analysis, and attachment functionality for documents, spreadsheets, PDFs, images, and receipts
4. To provide conversation-level insights from recent chat history including theme summarization, recommended actions, and next prompts
5. To implement secure authentication with two-factor authentication, organization-aware collaboration, and comprehensive audit logging
6. To create a workflow automation system supporting manual, scheduled, and event-driven execution
7. To develop an extensible plugin system with fail-closed permissions for marketplace-style integration capabilities
8. To ensure transparent AI decision-making through exposed workflow metadata and agent traces

1.4 Scope of the Project

This project encompasses the development of a local-first full-stack finance workspace with comprehensive features:

1.4.1 Frontend Scope

- React 18 single-page application with TypeScript
- Vite build system for development and production bundling
- 33 route-level page components covering all finance management functions
- Dual-theme system supporting light and dark modes
- Real-time updates through Server-Sent Events (SSE)
- File upload and management capabilities
- Chat interface with streaming AI responses

1.4.2 Backend Scope

- Express 5 API server with TypeScript
- MongoDB database with 48 Mongoose models
- 100+ REST API endpoints organized into versioned routes
- JWT authentication with HttpOnly cookies
- Google OAuth integration
- Two-factor authentication using TOTP
- Rate limiting and CSRF protection
- Background job processing with BullMQ
- Real-time event broadcasting

1.4.3 AI Core Scope

- Python FastAPI service for AI orchestration
- LangGraph workflow engine for multi-agent coordination
- Six specialist financial agents
- Provider failover across multiple LLM providers
- Vision/OCR capabilities for receipt processing
- Memory system for conversation personalization

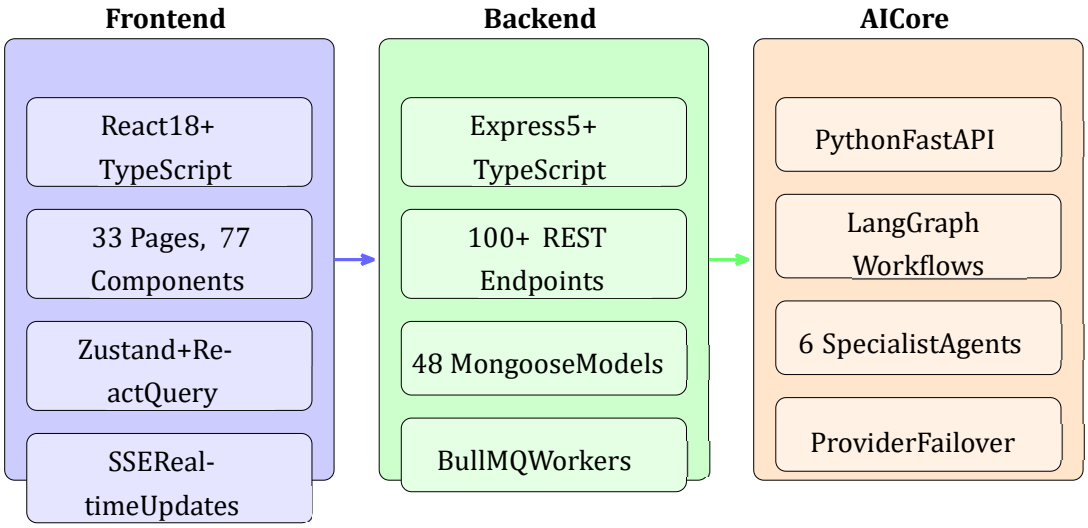


Figure2.Three-PillarSystemArchitectureScope

1.5 Key Contributions

This project makes several significant contributions to the field of personal finance management:

1. **Multi-Agent Financial Reasoning Framework**: A novel architecture using LangGraph to orchestrate specialist financial agents for comprehensive analysis covering budgeting, debt optimization, investment advice, and financial education.

2. **Transparent AI Decision Support**: Exposure of workflow metadata including agents involved, processing steps, and reasoning traces, enabling users to understand and trust AI recommendations.

3. **Integrated Finance-AI Platform**: Seamless combination of traditional finance operations (transactions, budgets, goals) with AI-powered insights within a unified product experience.

4. **Secure Extensibility Model**: Plugin system with fail-closed permissions balancing extensibility with security through manifest validation, permission sandboxing, and runtime isolation.

5. **Autopilot Workflow System**: Safe automation funnel with plan, simulate, approve, and execute stages ensuring human oversight of AI-recommended actions.

1.6 Organization of the Report

The remainder of this report is organized as follows:

Section 2: Related Work presents a comprehensive literature survey covering personal finance management systems, large language models in finance, multi-agent systems for decision support, and identifies gaps in existing approaches.

Section 3: Data and Methods describes the system architecture, including the frontend, backend, and AI Core components. It details the multi-agent workflow design, data models, security architecture, workflow automation system, and plugin system.

Section 4: Results and Discussions presents the implementation results, system features, API coverage, security implementations, performance characteristics, and discusses the strengths and limitations of the approach.

Section 5: Conclusion summarizes the project achievements, key contributions, and outlines directions for future work.

2 Related Work

2.1 Personal Finance Management Systems

The domain of personal finance management has evolved significantly over the past decade. Traditional applications like Mint, YNAB (You Need A Budget), and Personal Capital have established the foundation for transaction tracking and budget visualization [4]. However, these systems primarily focus on data aggregation and presentation without providing intelligent decision support.

2.1.1 Transaction Tracking Systems

Early personal finance applications focused primarily on transaction recording and categorization. Systems like Quicken and Microsoft Money pioneered desktop-based financial tracking in the 1990s, followed by web-based alternatives in the 2000s. These systems excelled at data organization but required users to manually interpret their financial data and make decisions
without systematic guidance.

2.1.2 Budget Visualization Tools

The emergence of mobile applications brought new approaches to budget visualization. Applications like YNAB introduced zero-based budgeting methodologies, while Mint popularized automatic transaction categorization through bank integrations. However, these tools still operate as passive information displays rather than active decision support systems.

2.1.3 Investment Tracking Platforms

Platforms like Personal Capital and Wealthfront combined personal finance tracking with investment management capabilities. While these systems offer portfolio analysis and rebalancing recommendations, they typically operate as isolated investment tools rather than comprehensive financial planning systems.

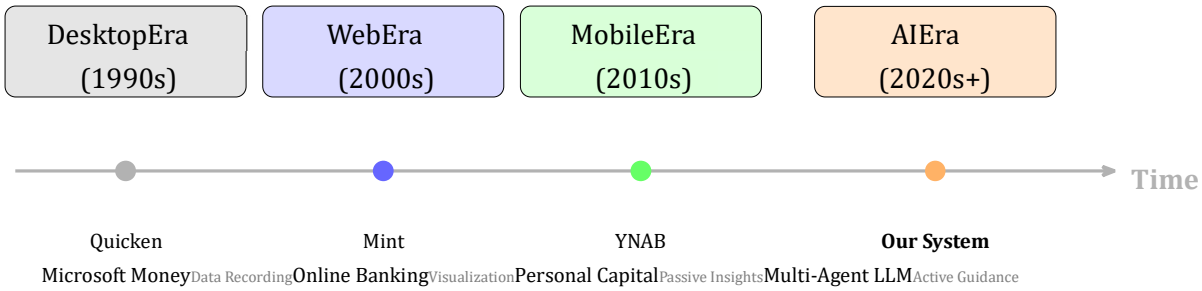


Figure 3. Evolution of Personal Finance Management Systems

2.2 Large Language Models in Finance

Recent advances in Large Language Models (LLMs) have opened new possibilities for financial applications. The development of transformer-based architectures has enabled models to understand and generate human-like text with remarkable accuracy [6].

2.2.1 Domain-Specific Financial Models

FinBERT and BloombergGPT represent specialized models trained on financial data for sentiment analysis and domain-specific tasks [1, 9]. FinBERT, fine-tuned on financial communication data, demonstrates superior performance on financial sentiment analysis compared to generalpurpose models. BloombergGPT, trained on a massive corpus of financial documents, shows strong capabilities in financial question answering and named entity recognition.

2.2.2 General-Purpose LLMs in Finance

The release of GPT-4 and similar large-scale models has enabled sophisticated financial reasoning capabilities [5]. These models can understand complex financial concepts, perform calculations, and generate explanations. However, they typically operate as single-agent systems without the structured workflow orchestration necessary for comprehensive financial planning.

2.2.3 Limitations of Current LLM Approaches

Despite their capabilities, current LLM applications in finance face several limitations:

- **Single-Agent Architecture**: Most implementations use a single model for all tasks, lacking specialization.
- **Context Limitations**: Token limits restrict the amount of financial history that can be processed.
- **Lack of Tool Integration**: Limited ability to perform actual calculations or access real-time data.
- **Opacity**: Black-box nature makes it difficult for users to understand the reasoning behind recommendations.

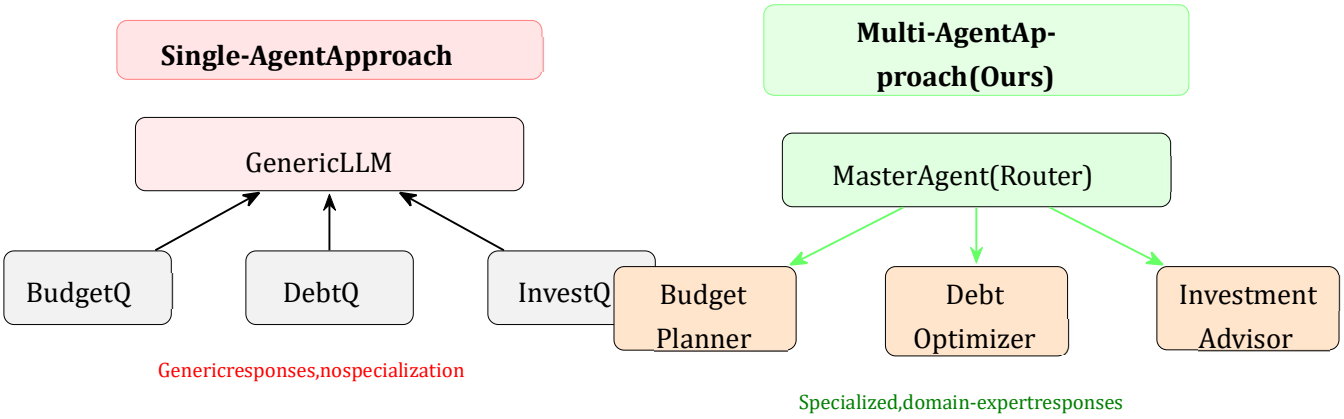


Figure 4. Single-Agent vs Multi-Agent Architecture Comparison

2.3 Multi-Agent Systems for Decision Support

Multi-agent systems have demonstrated effectiveness in complex decision-making scenarios. The LangGraph framework enables the construction of directed agent graphs for modular, testable, and composable reasoning [3].

2.3.1 Agent Orchestration Frameworks

Recent developments in agent orchestration have produced frameworks like AutoGen and CrewAI that enable multiple AI agents to collaborate on complex tasks [8]. These frameworks support role-based agent design, inter-agent communication, and task decomposition.

2.3.2 ReAct and Chain-of-Thought Approaches

The ReAct (Reasoning and Acting) paradigm combines reasoning traces with action execution, enabling models to interleave thought processes with tool usage [10]. Chain-of-thought prompting has shown significant improvements in complex reasoning tasks [7].

2.3.3 Application to Financial Reasoning

Multi-agent approaches are particularly valuable for financial reasoning where questions often combine several domains simultaneously, such as spending behavior, budget constraints, debt pressure, and investment tradeoffs. A specialized agent for each domain can provide more focused analysis than a single generalist agent.

2.4 Workflow Automation in Finance

Automation of financial tasks has traditionally been limited to simple rule-based systems like automatic bill payments and savings transfers. Modern approaches seek to enable more sophisticated automation while maintaining user control and trust.

2.4.1 Rule-Based Automation

Traditional financial automation relies on predefined rules: if-then conditions that trigger specific actions. While reliable, these systems lack flexibility and cannot adapt to changing circumstances without manual reconfiguration.

2.4.2 AI-Assisted Automation

Emerging approaches use AI to suggest automation rules based on observed patterns. However, most implementations lack the safety mechanisms necessary for financial operations, where incorrect actions can have significant consequences.

2.5 Comparison with Existing Approaches

Table 1 presents a comprehensive comparison of our approach with existing personal finance management systems.

Table 1
Comparison with Existing Personal Finance Systems

Feature	Mint	YNAB	Personal Capital	ChatGPT	Our System
Transaction Tracking	✓	✓	✓	–	✓
Budget Management	✓	✓	✓	–	✓
Investment Analysis	Limited	No	✓	Limited	✓
AI-Powered Insights	Limited	No	Limited	✓	✓
Multi-Agent Reasoning	No	No	No	No	✓
Workflow Automation	No	No	No	No	✓
Collaboration Features	No	Limited	No	No	✓
Plugin Ecosystem	No	No	No	No	✓
Local-First Architecture	No	No	No	No	✓
Transparent AI Reasoning	No	No	No	No	✓
Two-Factor Authentication	✓	✓	✓	✓	✓
Receipt OCR	Limited	No	No	✓	✓

2.6 Gap Analysis

The literature review reveals several gaps in existing personal finance management solutions:

1. **Lack of Structured AI Reasoning**: Existing AI-enabled finance apps use single-agent approaches without specialized financial domain expertise. This results in generic advice that may not account for the user's complete financial picture.

2. **Limited Automation**: Most systems lack workflow automation capabilities for scheduled and event-driven financial tasks. Users must manually execute repetitive operations.

3. **Poor Collaboration Support**: Family or team-based financial planning is poorly supported in existing tools. Most applications assume a single user without provision for shared financial management.

4. **No Explainability**: AI recommendations are often black-box without transparency into the reasoning process. This limits user trust and prevents informed decision-making.

5. **Limited Extensibility**: Closed ecosystems prevent third-party integrations and customizations. Users cannot extend functionality beyond what the platform provides.

6. **Inadequate Security for Shared Use**: Multi-user scenarios require sophisticated access control, audit logging, and data isolation that most systems do not provide.

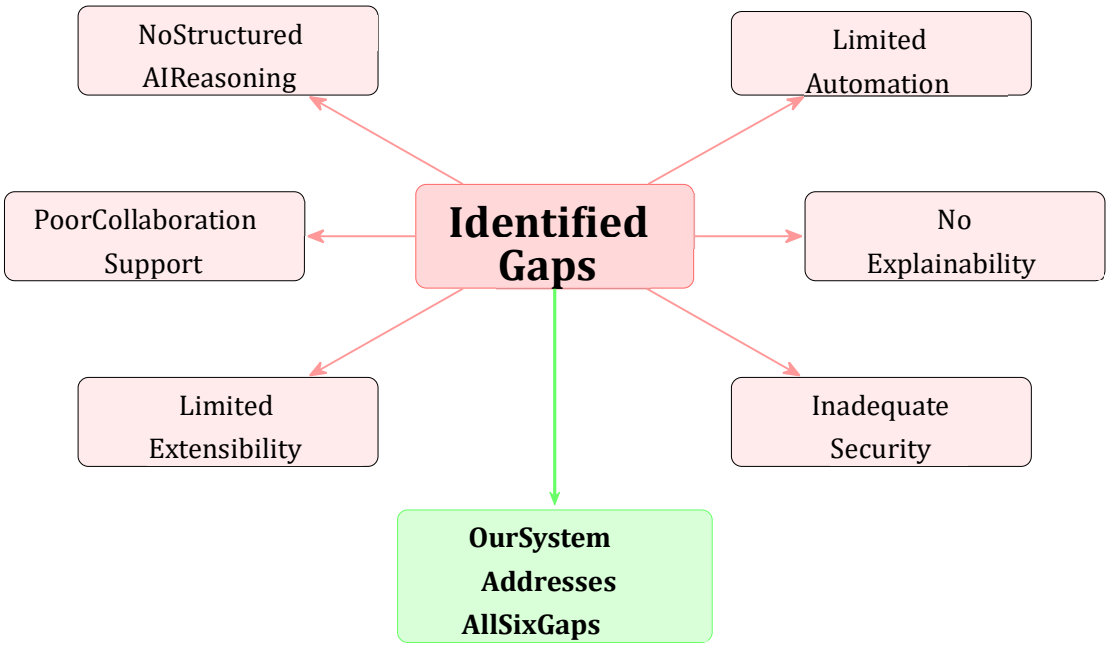


Figure 5. Gap Analysis Summary and Solution Coverage

Our project addresses these gaps through a multi-agent architecture with workflow metadata exposure, organization-aware collaboration, automated workflows with approval stages, and a plugin system for extensibility.

3 Data and Methods

3.1 System Architecture Overview

The system follows a modular multi-service architecture organized as a monorepo with three major runtime subsystems and one shared contracts package. This architecture enables independent development, testing, and scaling of each component while maintaining type safety across boundaries.

Table 2
System Subsystems Overview

Subsystem	**Language**	**Runtime**	**Purpose**
Client	TypeScript/React 18	Vite/Static Bundle	Single-page application for dashboard, chat, workflows
Server	TypeScript/Express 5	Node.js 18+	REST API, background workers, real-time SSE events
AI Core	Python 3.11+	FastAPI/LangGraph	Multi-agent financial intelligence engine
Contracts	TypeScript	Shared Package	OpenAPI specs and TypeScript contracts

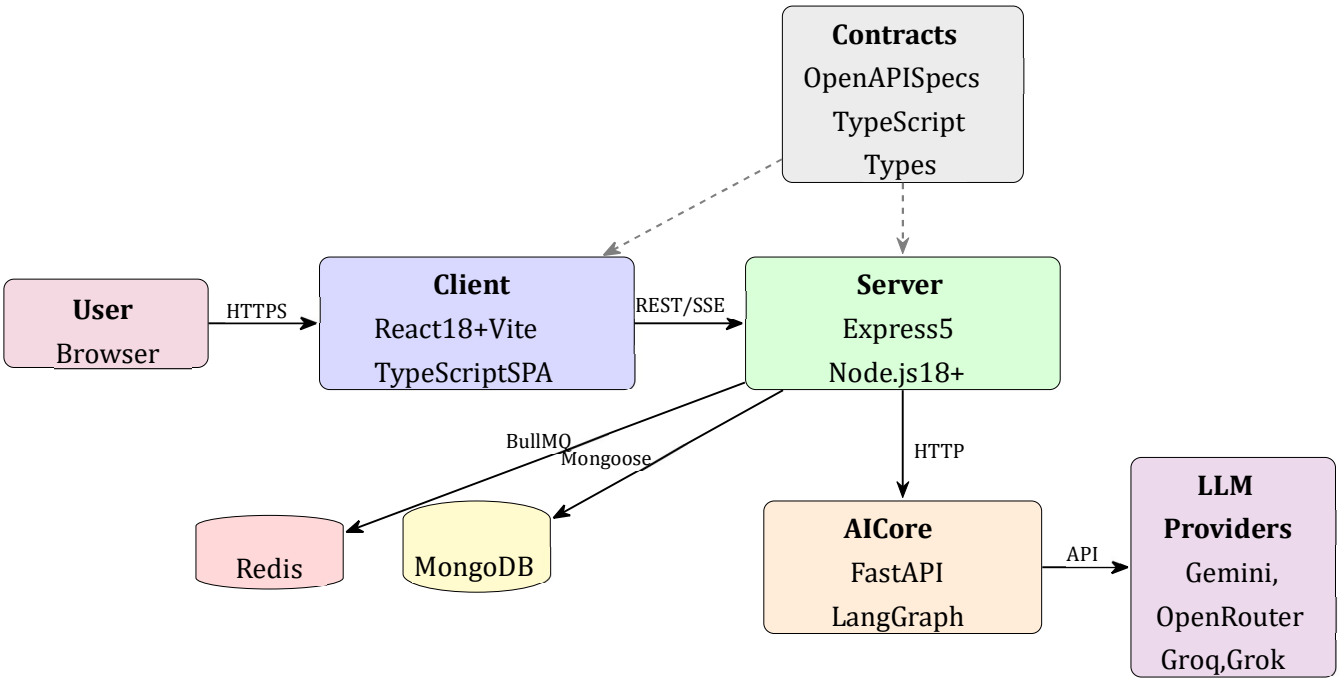


Figure6.High-LevelSystemArchitectureDiagram

3.2 Directory Structure

The repository follows a well-organized structure that separates concerns and facilitates development:

Listing 1: Repository Structure

```
personal-finance/
        client/ src/                    # React frontend
                components/             # 47 UI primitives + 30 feature components
                features/               # Feature modules (chat, journaling)
                hooks/                  # 13 custom React hooks
                lib/                    # API client layer & utilities
                pages/                  # 33 route-level page components
                stores/                 # 5 Zustand state stores
```

```
        types/              # Shared TypeScript interfaces
    package.json
server/                     # Express backend
    AI_Core/                # Python AI agent system
        agents/             # 6 specialist agents
        graph/              # LangGraph workflow & state
        tools/              # Agent tool definitions
        memory/             # Conversation memory
        vision/ src/        # OCR & image analysis

        config/             # Database, env, Redis, passport
        controllers/        # 45 route controllers
        middleware/         # 13 middleware modules
        models/             # 48 Mongoose models
        routes/                 # 15 route files (100+ endpoints)
        services/           # 50 business-logic services
        worker/             # BullMQ background workers
    package.json
packages/contracts/         # Shared OpenAPI specs
docs/                       # 33 documentation files
```

3.3 Frontend Architecture

The client application is built with React 18 and TypeScript, using Vite for development and bundling. The frontend architecture emphasizes component reusability, type safety, and efficient state management.

3.3.1 Technology Stack

Table 3
Frontend Technology Stack

Layer	Technology	Purpose
Framework	React 18	Component-based UI development

Build Tool	Vite 5+	Fast development server and optimized builds
Language	TypeScript 5+	Type safety and developer experience
Routing	Wouter	Lightweight client-side routing
State Management	Zustand	Minimal boilerplate state stores
Data Fetching	React Query 5+	Server state caching and synchronization
Styling	Tailwind CSS	Utility-first CSS framework
UI Components	Radix UI	Accessible component primitives
Forms	React Hook Form + Zod	Form state management and validation
Charts	Recharts	Data visualization
Animations	Framer Motion	Smooth transitions and animations
Icons	Lucide React	Consistent iconography

3.3.2 State Management Strategy

The application uses a hybrid state management approach:

React Query handles all server state, including:

- Transaction data with automatic caching
- User profile and authentication state
- Organization settings and membership
- Chat sessions and message history

Zustand manages UI/session state through five stores:

- chatStore: Chat sessions, messages, streaming state
- aiStore: AI command bar state, pending actions

- appDialogStore: Global dialog/modal visibility
- commandBarStore: Command palette state • orgStore: Active organization context

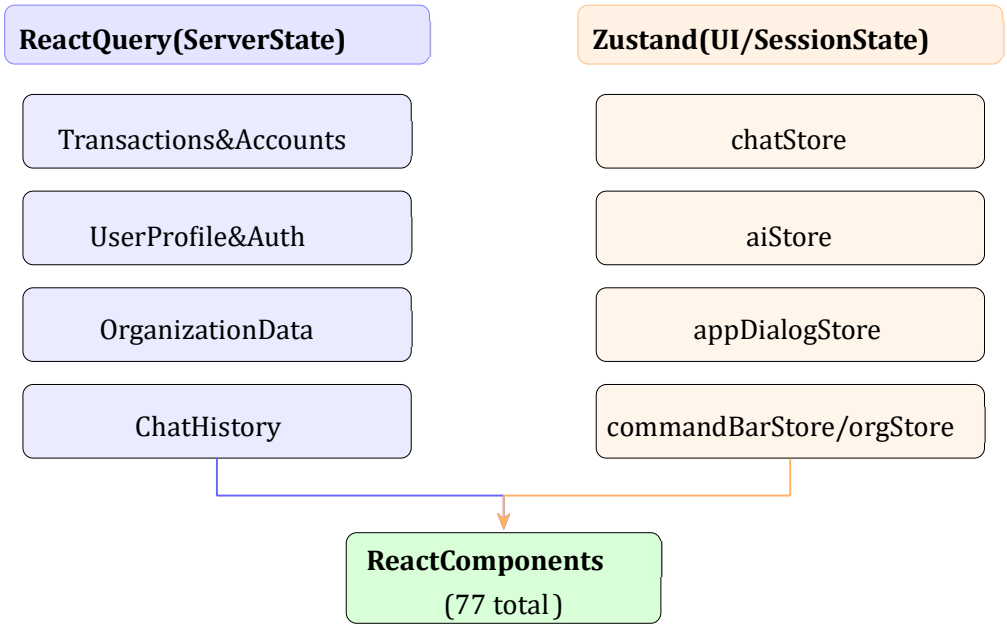


Figure 7. Hybrid State Management Architecture

3.3.3 Routing Architecture

The application implements protected and public routes using Wouter:

Table 4
Route Categories

Category	Routes	Examples
Public	5	/login, /register, /verify-email, /accept-invite
Protected (App Layout)	24	/dashboard, /transactions, /workflows, /tasks
Protected (Chat Layout)	2	/chat, /chat/:sessionId

Content	4	/blogs, /blogs/:slug, /growthstories

3.4 Backend Architecture

The Express 5 server with TypeScript serves as the control plane of the system, handling authentication, authorization, data persistence, and orchestration of AI requests.

3.4.1 Middleware Stack

The server implements a comprehensive middleware stack applied in order:

Table 5 Middleware Stack (Application Order)

#	**Middleware**	**Purpose**
1	CORS	Restrict cross-origin requests to allowed origins
2	Helmet	Set security HTTP headers (CSP, X-Frame, etc.)
3	Security Headers	Additional headers: HSTS, Permissions-Policy
4	JSON Parser	Parse JSON bodies with configurable size limit
5	NoSQL Sanitizer	Strip $ and . keys to prevent injection
6	Cookie Parser	Parse cookies with signed-cookie secret
7	Passport	Initialize JWT/OAuth strategies
8	Request Context	Generate X-Request-Id, start request timer
9	Response Context	Inject request id and response timing

10	Legacy API Deprecation	Add X-API-Deprecation headers
11	HTTP Logger	Pino HTTP request/response logging
12	Metrics	Prometheus request counters and histograms
13	Optional JWT Auth	Attempt JWT decode; attach req.user
14	Org Context	Resolve org from JWT or API key
15	Rate Limiter	Global rate limit on /api routes
16	CSRF Protection	Double-submit cookie CSRF validation

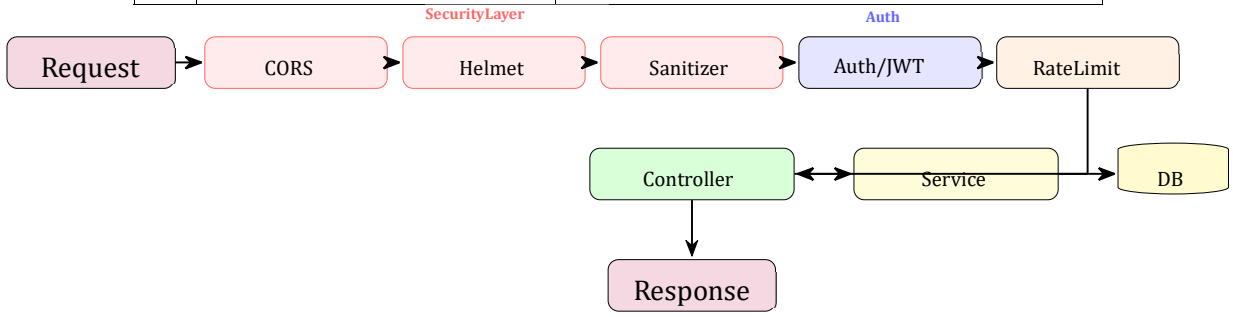


Figure8.RequestFlowThroughMiddlewarePipeline

3.5 AI Core Architecture

The AI Core is a Python FastAPI service that implements multi-agent financial reasoning using LangGraph for workflow orchestration.

3.5.1 Agent System

The AI Core implements six specialist agents, each focused on a specific financial domain:

Table 7
Specialist Agents

Agent	**Responsibilities**
Master Agent	Query classification, agent routing, response synthesis

Income/Expense Analyzer	Transaction pattern analysis, spending trends, anomaly detection, savings rate calculation
Budget Planner	Category-level budget allocations, methodology support (50/30/20, zero-based), optimization suggestions
Investment Advisor	Portfolio allocation recommendations, risk-based suggestions, rebalancing strategies
Debt Optimizer	Repayment strategy comparison (avalanche vs snowball), payoff timeline calculation, consolidation opportunities
Financial Educator	Financial literacy explanations, concept teaching, minilessons with examples

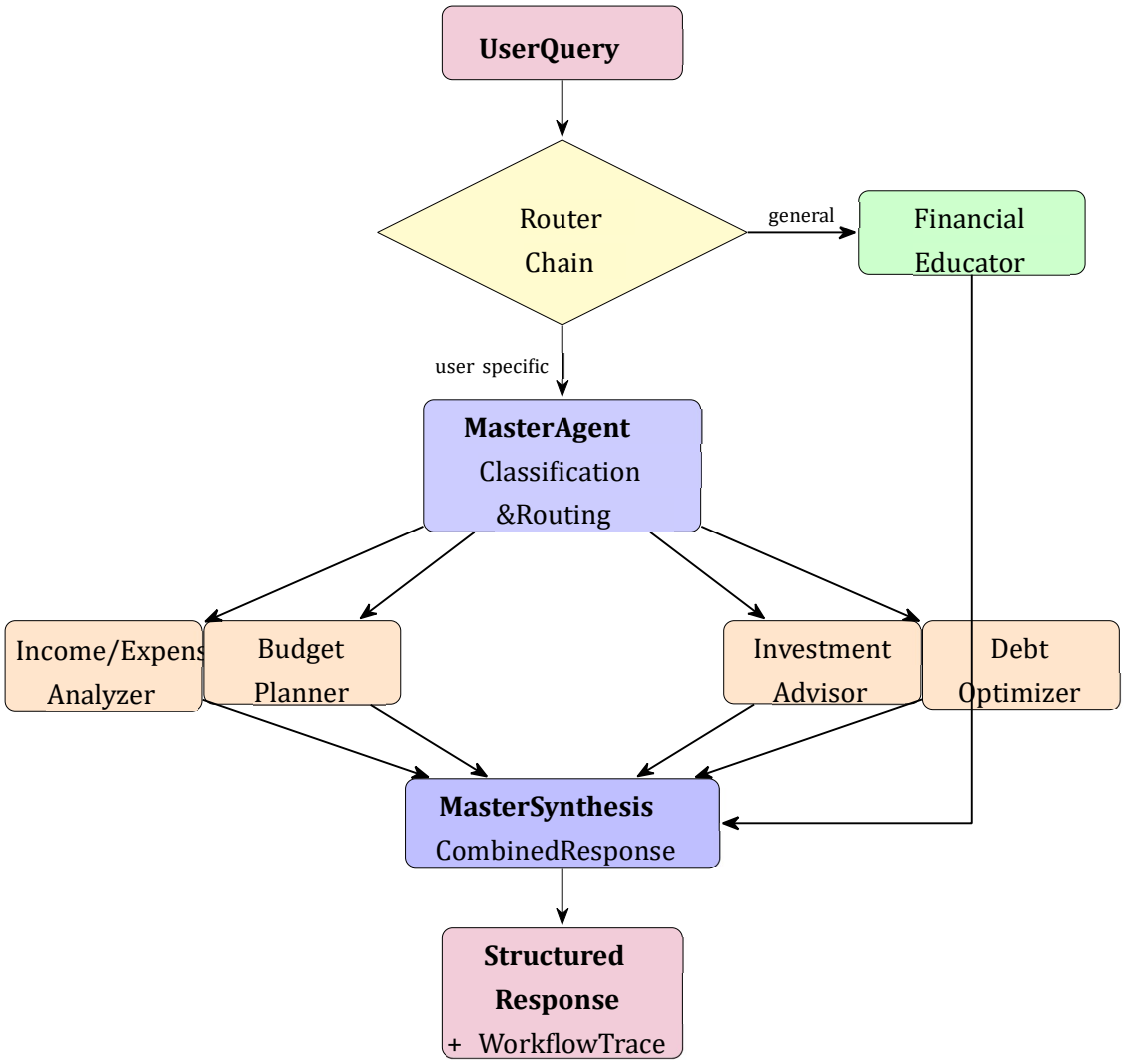


Figure 9. Multi-Agent LangGraph Workflow Architecture

Query Routing System

The AI Core uses a two-stage routing system:

1. **Router Chain**: Gemini-powered classification of queries as user specific or general
    - user specific queries receive full user profile context
    - general queries route directly to Financial Educator
2. **Master Agent Delegation**: Further routes to specific specialists based on intent analysis

3.5.2 Provider Failover

The AI Core implements provider failover with the following chain:

1. Gemini (Primary)
2. OpenRouter
3. Groq
4. Grok
5. Together

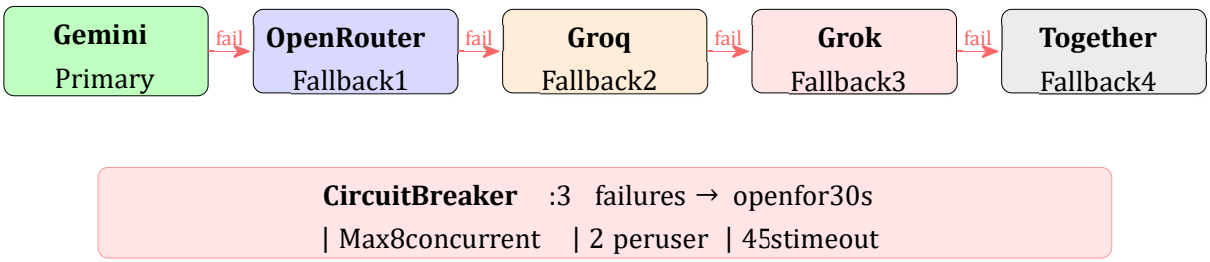


Figure 10. LLM Provider Failover Chain with Circuit Breaker

3.5.3 Circuit Breaker Pattern

The server implements a circuit breaker for AI Core communication:

Table 8
Circuit Breaker Configuration

Configuration	Default	Description
AI CORE CIRCUIT _FAILURE THRESHOLD	3	Failures before circuit opens
AI CORE CIRCUIT _OPEN _MS	30000	Duration circuit stays open
AI CORE HEALTH CACHE MS	5000	Health response cache time
AI CORE TIMEOUT _MS	45000	Request timeout
AI CORE MAX _CONCURRENCY	8	Global concurrent request limit
AI CORE MAX _CONCURRENCY PER USER	2	Per-user concurrent limit

3.6 Data Model Design

The system uses MongoDB with Mongoose, implementing 48 models organized into functional categories.

3.6.1 Core Design Principles

1. **Multi-tenancy**: Most collections are org-scoped via orgId field
2. **Ownership**: userId or createdByUserId fields indicate actor/owner
3. **Timestamps**: Automatic createdAt and updatedAt via Mongoose
4. **Idempotency**: idempotencyKey fields for safe retry operations
5. **Provenance**: Source metadata tracking origin of records

3.6.2 Model Categories

Table 9
Database Model Categories

Category	Models	Key Models
Identity & Org	4	User, Organization, OrgMember, OrgInvite
Financial Data	8	Transaction, Account, BudgetAllocation, RecurringRule, Month-Close
AI & Chat	7	ChatSession, ChatMessage, AgentOutput, MemoryRecord, AiResponseCache
Tasks & Workflows	5	Task, Workflow, WorkflowRun, ToolExecution, AutopilotRun
Collaboration	4	Comment, Notification, Share-Link, DomainEvent
Billing	8	Subscription, Entitlement, UsageEvent, UsageLedger, Credit-Grant
Marketplace	4	MarketplacePlugin, PluginInstall, IntegrationConnection
Observability	4	AuditEvent, AuditLog, Feature-Flag, ExportJob
Content	4	BlogPost, GrowthStory, JournalEntry, Receipt

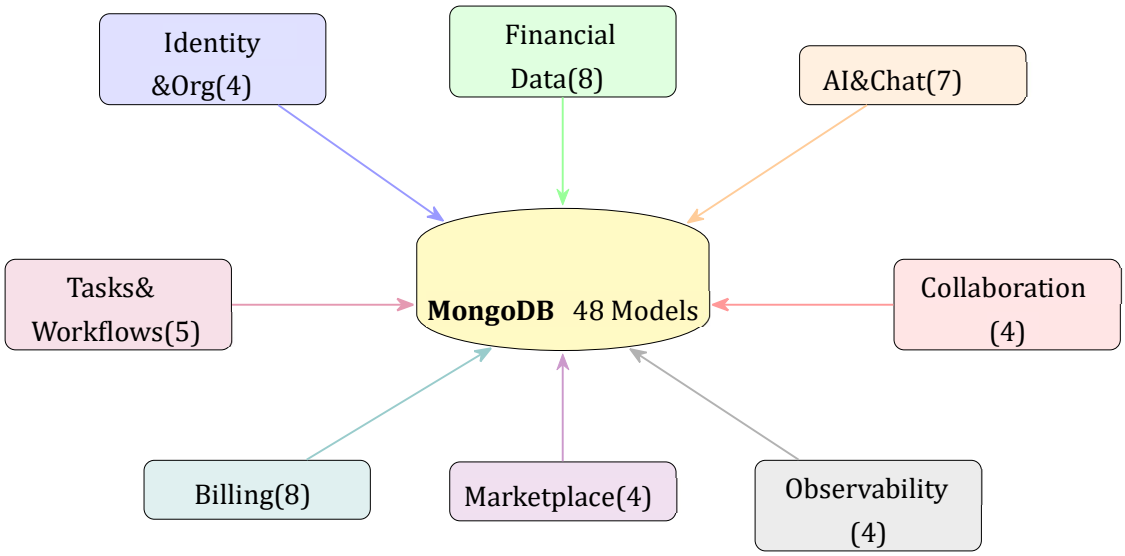


Figure 11. Database Model Organization (48 Models across 9 Categories)

3.7 Security Architecture

The system implements defense-in-depth security with OWASP API Security Top 10 coverage.

Layer 1: Transport Security – HSTS, CORS, Secure Cookies, CSP Headers

Layer 2: Authentication – JWT, OAuth, TOTP 2FA, API Keys

Layer 3: Authorization – RBAC, Org Isolation, Scope Enforcement

Layer 4: Input Validation – Zod, NoSQL Sanitizer

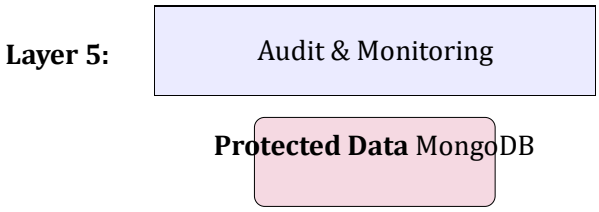


Figure 12. Defense-in-Depth Security Architecture

3.7.1 Authentication Mechanisms

Table 10
Authentication Methods

Method	Implementation
JWT Auth	HttpOnly cookies with configurable expiry, token refresh support
Google OAuth	OAuth 2.0 integration with automatic account linking
API Keys	Scoped keys with usage tracking, quota enforcement, expiration
Two-Factor Auth	RFC 6238 TOTP with SHA-1, 6 digits, 30s period, backup codes

3.7.2 Security Controls

Table 11
Security Control Matrix

Control	Implementation
Transport	HSTS (production), secure cookies, CORS origin allowlist
Authorization	Org-scoped data isolation, RBAC via entitlements, API key scope enforcement
Input Validation	Zod schema validation, NoSQL injection sanitizer, request size limits
Rate Limiting	Per-org/user/IP limits, stricter auth endpoint limits
Brute Force	Account lockout after 5 failed attempts in 15-minute window
CSRF	Double-submit cookie pattern, configurable per environment
Audit	26 security event types, 365-day retention, immutable records
Plugin Sandbox	Fail-closed permission enforcement, high-risk permission flagging
Headers	Helmet + custom headers (X-Content-Type-Options, Permissions-Policy)

3.7.3 Action Types

Table 13
Workflow Action Types

Action	Capabilities
create task	Generate tasks with bucket (7/30/365 days), title, why, steps, priority, kind

send notification	Email or in-app notifications with configurable subject and message
export report	Generate transactions csv or monthly summary pdf exports

3.7.4 Autopilot Lifecycle

The Autopilot system implements a safe automation funnel:

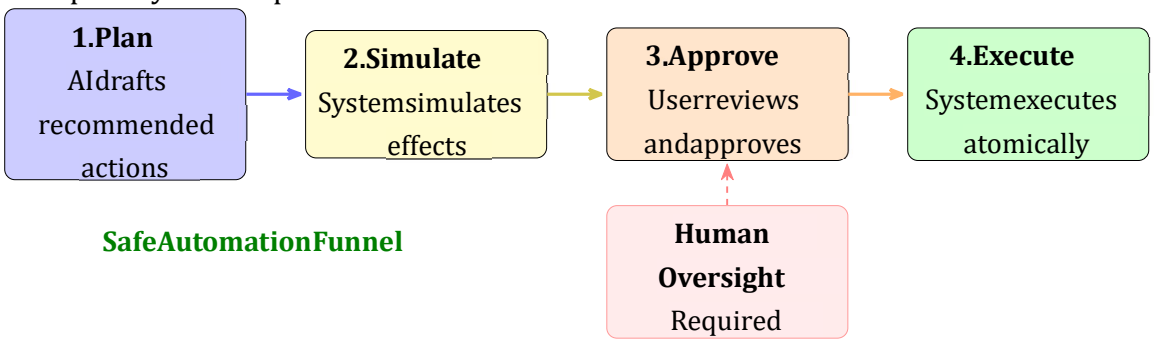


Figure 13. Autopilot Lifecycle: Safe Automation with Human Oversight

3.8 Plugin System Architecture

The plugin system enables extensibility through a sandboxed, permission-enforced runtime.

3.8.1 System Components

Table 14
Plugin System Components

Component	Responsibility
Plugin Manager	Lifecycle handling: install, update, uninstall, list
Permission Sandbox	Fail-closed permission evaluation and enforcement
Permission Middleware	Request-time permission checks, manifest validation
Runtime Client	HTTP communication with external plugin services

3.8.2 Permission Scopes

Table 15 Plugin
Permission Scopes

Permission	Description	Risk
transactions:read	Read transaction history	Normal
transactions:write	Create/update/delete transactions	High
budgets:read	Read budget allocations	Normal
budgets:write	Modify budget allocations	High
profile:read	Read user profile	Normal
profile:write	Modify profile	High
workflows:create	Create workflows	Normal
workflows:execute	Execute workflow actions	High
notifications:send	Send user notifications	Normal
exports:write	Generate data exports	Normal

a

3.9 Observability Stack

The system implements comprehensive observability through logging, metrics, and tracing.

Table 16
Observability Stack

Layer	Tool	Configuration
Logging	Pino (structured JSON)	Stdout / forwarded to log aggregator
Metrics	Prometheus (prom-client)	GET /api/metrics (requires METRICS TOKEN)

Tracing	OpenTelemetry	Auto-instrumented HTTP, Mongo, Redis
Alerting	Grafana / PagerDuty	Built on metrics + log queries

3.10 API Design

The API follows RESTful principles with versioned endpoints and consistent response formats.

3.10.1 Versioning Strategy

- Canonical API surface: /api/v1/*
- Legacy routes: /api/* (deprecated, includes X-API-Deprecation headers)
- All new features target /api/v1 exclusively

3.10.2 Error Response Format

Listing 3: Error Response Shape

```
{
  "message": "Human-readable description",
  "code": "VALIDATION_ERROR",
  "details": [
    { "field": "email", "message": "Invalid email format" }
  ],
  "request_id": "req_abc123"
}
```

4 Results and Discussions

4.1 System Implementation

The Smart Personal Finance Management system has been successfully implemented with all planned features. The system demonstrates the viability of combining traditional finance tooling

with structured AI reasoning in a production-ready architecture.

4.1.1 Implementation Statistics

Table 17
Implementation Statistics

Component	Count
Frontend Pages	33
UI Components	77 (47 primitives + 30 feature)
Custom React Hooks	13
Zustand Stores	5
Backend Controllers	45
REST API Endpoints	100+
Mongoose Models	48
Business Services	50
Middleware Modules	13
AI Agents	6
Documentation Files	33
Test Files	34

Count

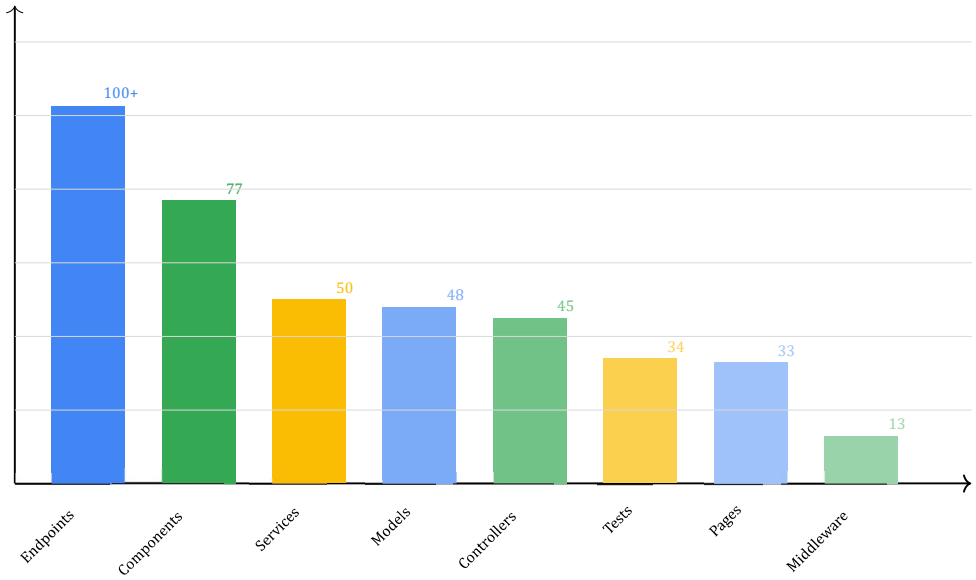


Figure 15. Implementation Statistics Visualization

4.1.2 User Interface Implementation

The dashboard provides a chat-first experience prioritizing conversational workflows. Key UI implementations include:

- **Dual-Theme System**: Clean white light theme and pure black dark theme with automatic system preference detection
- **Files Workspace**: Dedicated page for document management with AI analysis capabilities
- **Chat File Attachments**: Support for PDFs, spreadsheets, images, code files, and documents
- **Conversation Insights Panel**: Real-time display of themes and recommended actions
- **Lazy Loading**: All 33 pages use React.lazy() for optimized initial bundle size
- **Responsive Design**: Full mobile and desktop support through Tailwind CSS

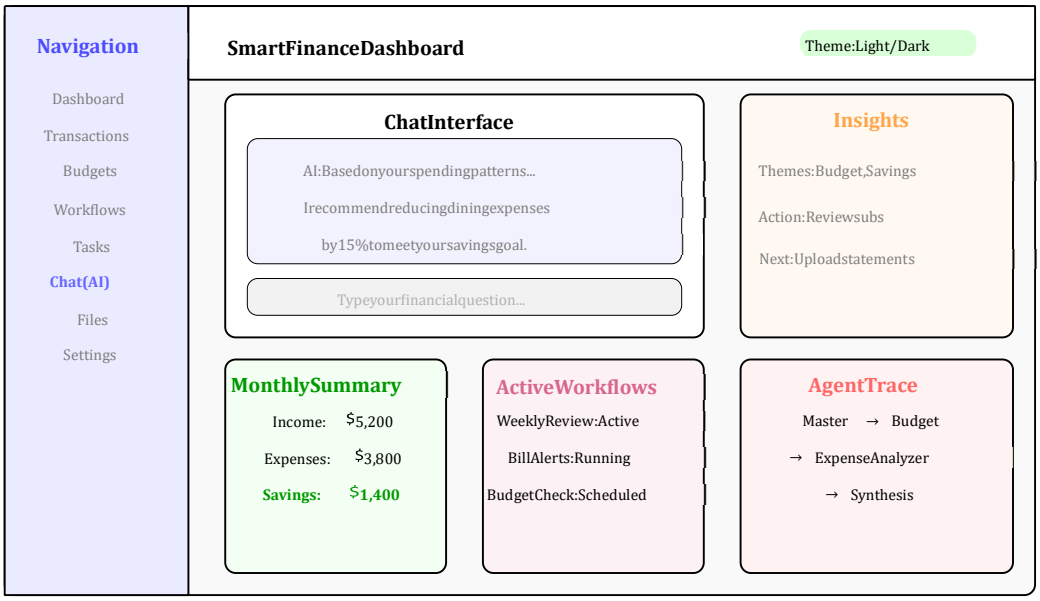


Figure 16. Dashboard UI Layout with Chat-First Design

4.2 Multi-Agent Response Analysis

The multi-agent system produces structured responses with full transparency into the reasoning process.

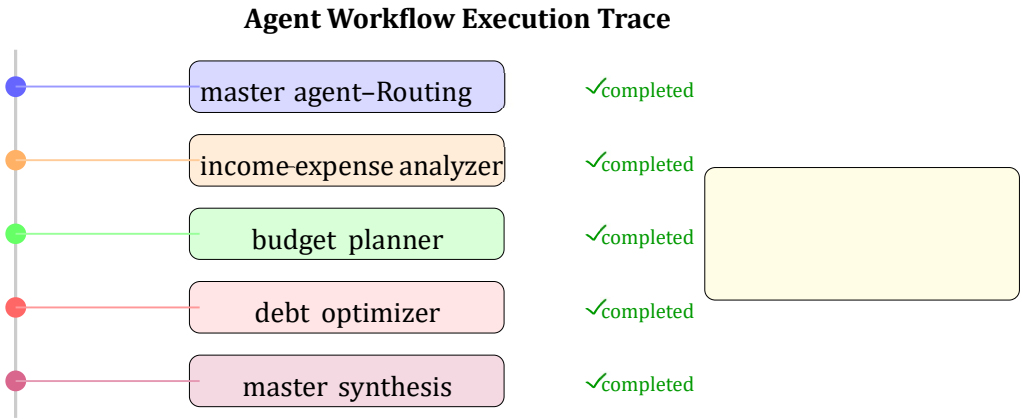


Figure 17. Agent Workflow Execution

Trace Visualization

4.2.1 Conversation Insights Example

Listing 5: Sample Conversation Insights

Conversation pulse: You are focused on reducing monthly cash pressure while building a disciplined savings habit.

Repeated themes: debt payoff, budget tightening, emergency fund planning.

Recommended next moves:

- Review subscriptions for potential cancellations
- Set a weekly spending cap for discretionary categories
- Upload recent statements for deeper analysis

Suggested next prompt: Compare my last 3 months of expenses and identify categories I can cut first.

4.3 API Coverage Analysis

The system provides comprehensive API coverage with over 100 REST endpoints organized into functional groups.

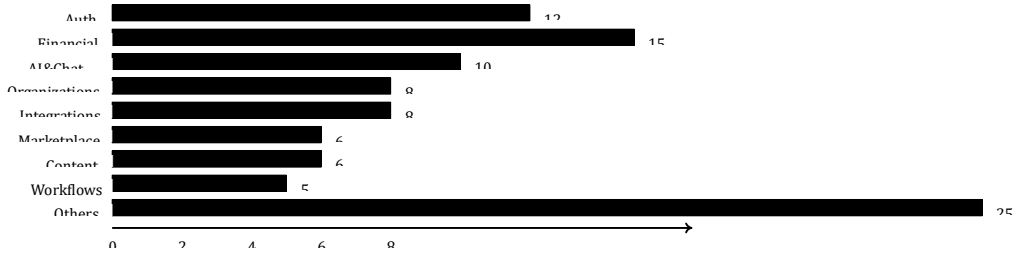
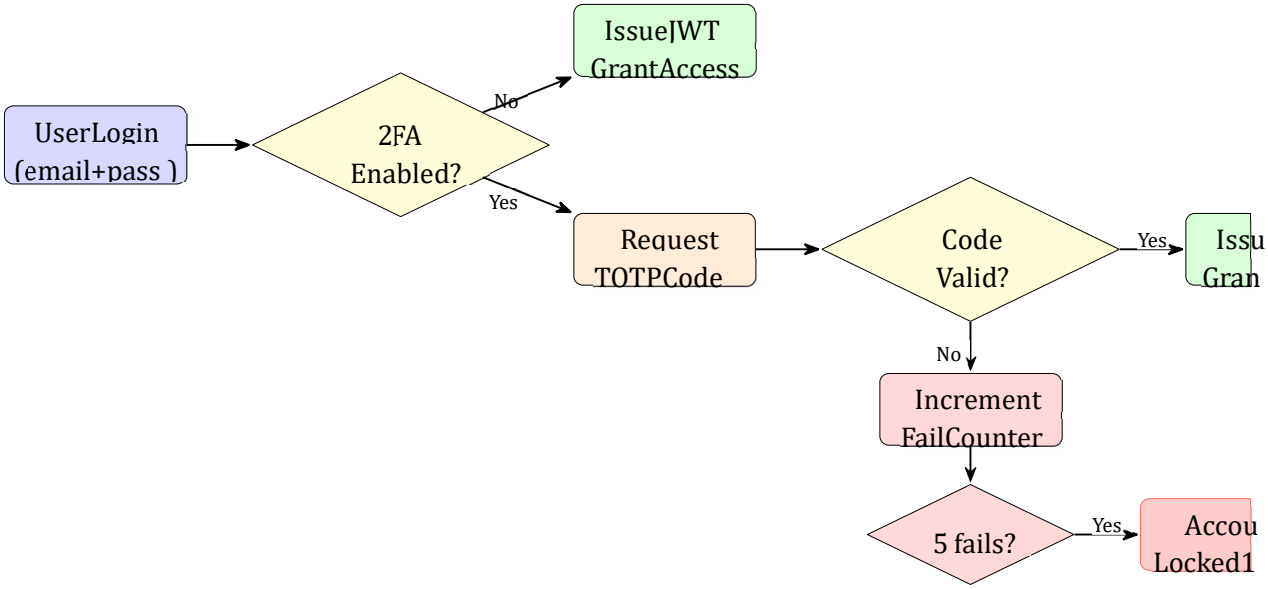


Figure 18. API Endpoint Distribution by Category



4.4 Security Implementation Results

The implemented security features provide comprehensive protection across all layers.

4.4.1 Two-Factor Authentication

- **Standard**: RFC 6238 TOTP with SHA-1 algorithm
- **Configuration**: 6 digits, 30-second period
- **Clock Drift**: ±1 period tolerance
- **Backup Codes**: 8 codes, SHA-256 hashed for storage
- **Audit**: All 2FA state changes logged
- **Window**: 15 minutes
- **Lockout Duration**: 15 minutes
- **Scope**: Email + IP combination
- **Reset**: Automatic after lockout expires; cleared on successful login

4.4.2 HTTP Security Headers

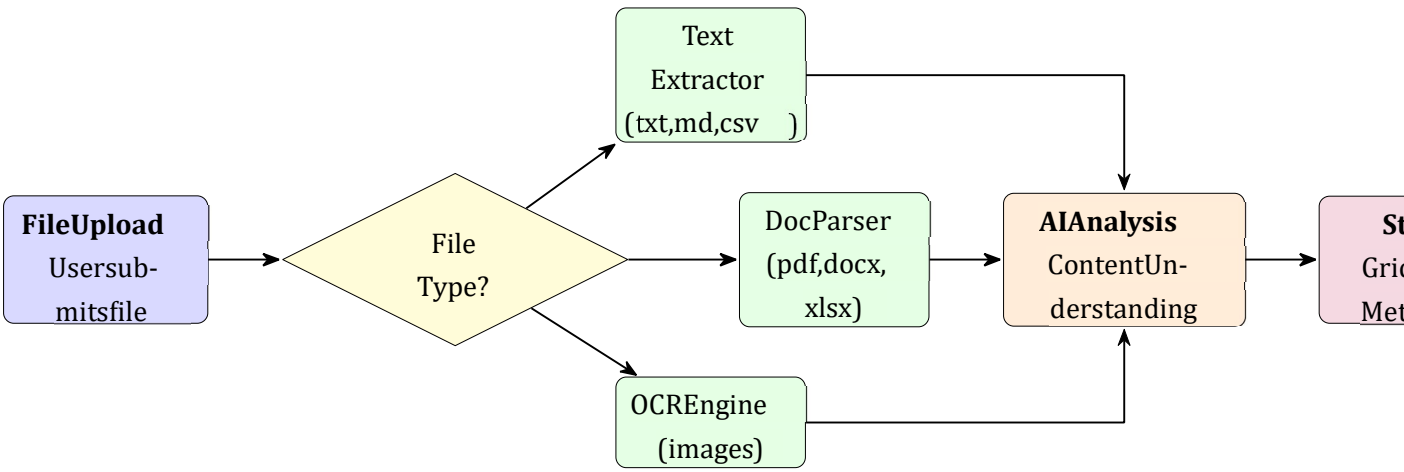
Table 19
Implemented Security Headers

Header	Value
X-Content-Type-Options	nosniff
X-Frame-Options	DENY
Referrer-Policy	strict-origin-when-cross-origin
Permissions-Policy	camera=(), microphone=(), geolocation=(), payment=(), usb=()
Cache-Control	no-store, no-cache, must-revalidate
Strict-Transport-Security	max-age=31536000; includeSubDomains; preload (production)
Content-Security-Policy	Restrictive policy via Helmet

4.5 File Processing Capabilities

The system supports comprehensive file handling with text extraction and AI analysis:

Table 20



Supported File Types and Processing

Type	Library	Capabilities

Plain Text	Native	Direct text extraction
Markdown	Native	Markdown parsing and rendering
Code Files	Native	Syntax highlighting, analysis
CSV	Native	Column parsing, data extraction
JSON/XML/YAML	Native	Structure parsing
Excel (xlsx)	xlsx library	Sheet extraction, data parsing
PDF	pdf-parse	Text extraction, page parsing
DOCX	mammoth	Document text extraction
Images	AI Core OCR	Text recognition, receipt parsing

Figure 20. File Processing Pipeline with Type Detection and AI Analysis

4.6 Performance Characteristics

The system implements several performance optimizations:

4.6.1 Caching Strategy

- **AI Response Cache**: Configurable TTL for repeated queries
- **React Query**: 30-second stale time for client-side caching
- **Dashboard Cache**: Precomputed summaries to reduce query load
- **Health Check Cache**: 5-second cache for AI Core health status

4.6.2 Concurrency Management

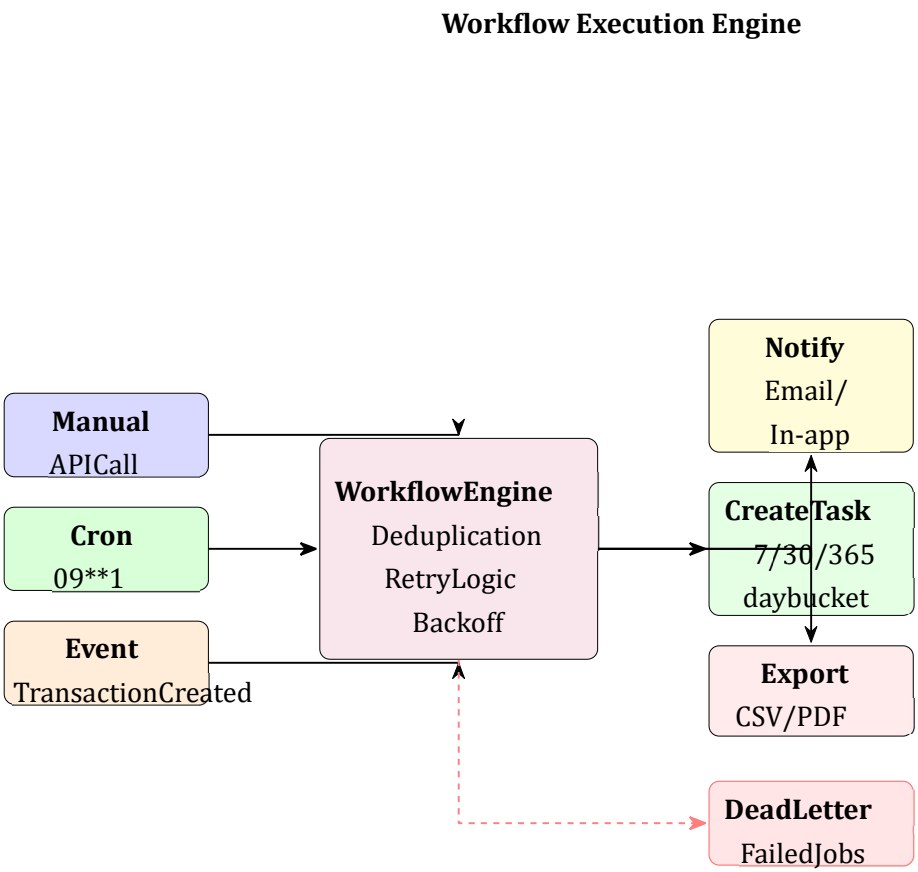


Figure 21. Workflow Execution Engine with Trigger Types and Actions

4.7 Discussion of Results

4.7.1 Strengths of the Approach

1. **Multi-Agent Architecture**: The specialist agent approach provides more focused and accurate financial analysis by decomposing complex queries into domain-specific tasks. Each agent can be optimized independently for its domain.

2. **Transparency**: The exposure of workflow metadata (agents involved, provider used, trace information) builds user trust by making the reasoning process inspectable. Users can understand why specific recommendations were made.

3. **Integration of Finance and AI**: The system is not just an AI chatbot bolted onto a finance UI. The architecture combines operational product features with a dedicated AI workflow that understands auth, org context, financial records, workflows, and automation state.

4. **Collaboration Support**: Real financial planning often involves multiple stakeholders. The organization-aware architecture with invites, comments, and activity tracking supports team-based financial management effectively.

5. **Extensibility**: The plugin system with fail-closed permissions enables third-party extensions without compromising security. The marketplace model supports ecosystem growth.

6. **Safe Automation**: The autopilot lifecycle (plan → simulate → approve → execute) ensures human oversight of AI-recommended actions, preventing unintended financial operations.

4.7.2 Comparison with Initial Objectives

Table 21
Objective Achievement Analysis

Objective	Status	Evidence
Multi-agent AI reasoning	Achieved	6 specialist agents implemented
Chat-first finance assistant	Achieved	Streaming chat with file attachments
File upload and analysis	Achieved	9+ file types supported
Conversation insights	Achieved	Theme extraction and recommendations
Secure authentication	Achieved	JWT, OAuth, 2FA implemented
Workflow automation	Achieved	Manual, cron, event triggers
Plugin ecosystem	Achieved	Marketplace with permissions
Transparent AI decisions	Achieved	Full workflow traces exposed

4.7.3 Limitations

Current limitations of the implementation include:

1. **Local Optimization**: The system is optimized for local demonstration and development rather than production-scale deployment. Additional hardening would be required for production use.

2. **Binary File Extraction**: Some binary file formats are stored without deep text extraction, limiting AI analysis capabilities for certain file types.

3. **Bank Connectivity**: Real-time bank connectivity requires additional integration work with financial data aggregators like Plaid or Yodlee.

4. **Plugin Runtime**: The plugin runtime requires external service deployment, adding operational complexity.

5. **Mobile Application**: The current implementation is web-only; mobile applications would require additional development.

4.7.4 Comparison with Related Work

Compared to existing solutions discussed in the literature review:

- **vs. Mint/YNAB**: Our system provides AI-powered insights and workflow automation that these passive tracking tools lack.

- **vs. ChatGPT**: Our multi-agent approach provides specialized financial analysis rather than general-purpose conversation. The integration with actual financial data enables contextual recommendations.

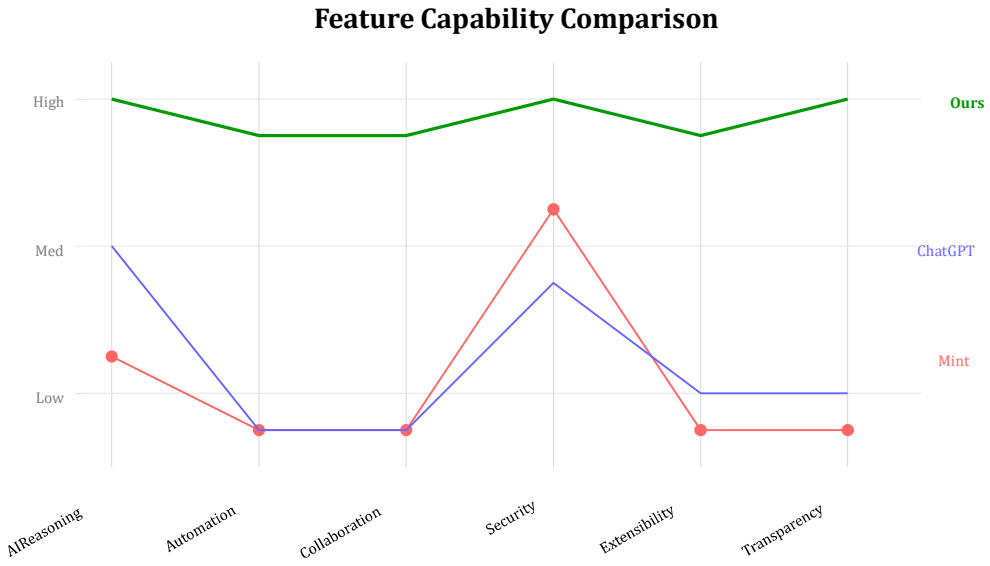


Figure 22. Feature Capability Comparison Across Systems

5 Conclusion

5.1 Summary of Achievements

This project successfully developed a Smart Personal Finance Management system using LLMBased Decision Support. The system addresses the fundamental limitation of existing personal finance applications that stop at passive data recording by providing intelligent, actionable guidance through multi-agent AI reasoning.

5.1.1 Technical Achievements

1. **Full-Stack Architecture**: Successfully implemented a three-tier architecture with React frontend (33 pages, 77 components), Express backend (100+ endpoints, 48 models), and Python AI Core (6 specialist agents).

2. **Multi-Agent System**: Developed a LangGraph-based multi-agent workflow with transparent execution traces, enabling users to understand the reasoning behind recommendations.

3. **Comprehensive Security**: Implemented defense-in-depth security including JWT authentication, two-factor authentication, CSRF protection, rate limiting, and comprehensive audit logging with 26 event types.

4. **Workflow Automation**: Created a flexible workflow engine supporting manual, scheduled
(cron), and event-driven triggers with idempotent execution and dead-letter queue handling.

5. **Plugin Ecosystem**: Developed an extensible plugin system with fail-closed permissions, manifest validation, and runtime isolation for secure third-party extensions.

6. **File Processing**: Implemented support for 9+ file types including PDF, Excel, images, and documents with text extraction and AI analysis capabilities.

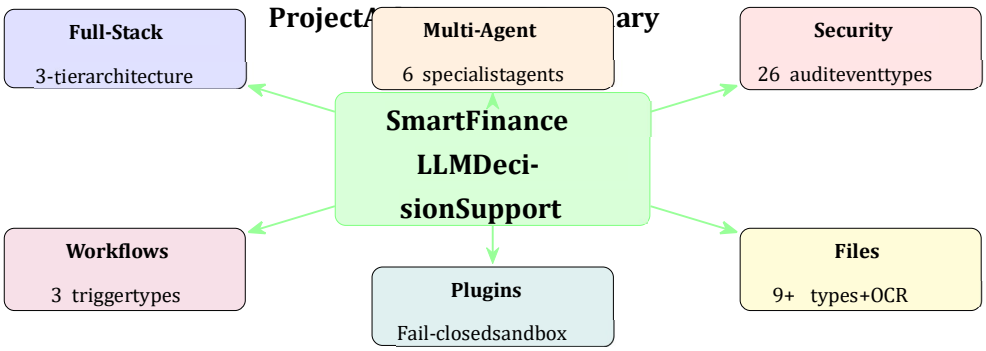


Figure23.ProjectAchievementSummary

5.1.2 Product Achievements

1. **Chat-First Experience**: Prioritized conversational workflows in the dashboard design, making AI assistance the primary interaction mode.

2. **Conversation Insights**: Implemented real-time theme extraction and recommendation generation from chat history.

3. **Collaboration Features**: Enabled organization-aware financial planning with invites, comments, activity feeds, and shared workflows.

4. **Safe Automation**: Implemented the autopilot lifecycle (plan → simulate → approve → execute) ensuring human oversight of AI-recommended actions.

5.2 Key Contributions

This project makes the following contributions to the field of AI-powered personal finance management:

1. **Multi-Agent Financial Reasoning Framework**: A novel architecture demonstrating how specialist AI agents can be orchestrated using LangGraph to provide comprehensive financial analysis that single-agent approaches cannot match.

2. **Transparent AI Decision Support**: A reference implementation showing how workflow metadata exposure can build user trust in AI recommendations through inspectable reasoning processes.

3. **Integrated Finance-AI Platform**: A demonstration of how traditional finance operations can be seamlessly combined with AI-powered insights within a unified product experience.

4. **Secure Extensibility Model**: An architecture pattern for plugin systems that balances extensibility with security through fail-closed permissions and runtime isolation.

5. **Safe Automation Paradigm**: The autopilot lifecycle pattern as a template for AIassisted automation that maintains human oversight of consequential actions.

5.3 Lessons Learned

Throughout the development process, several important lessons emerged:

1. **Multi-Agent Complexity**: Coordinating multiple AI agents requires careful attention to state management, error handling, and fallback behavior. The circuit breaker pattern proved essential for resilience.

2. **Security-First Design**: Building security controls from the beginning is significantly easier than retrofitting them. The org-scoped data model prevented many potential issues.

3. **Type Safety Value**: End-to-end TypeScript with shared contracts between frontend and backend dramatically reduced integration issues and improved developer productivity.

4. **Observability Importance**: Comprehensive logging, metrics, and tracing proved invaluable for debugging AI-related issues where behavior is less deterministic than traditional software.

5.4 Future Work

Several directions for future enhancement have been identified:

5.4.1 Near-Term Enhancements

- **Bank Integration**: Integration with financial data aggregators (Plaid, Yodlee) for automatic transaction import
- **Mobile Application**: Native iOS and Android applications for on-the-go access
- **Advanced Visualizations**: Interactive charts and customizable dashboard widgets
- **Voice Interface**: Voice-based financial queries for hands-free interaction

5.4.2 Medium-Term Enhancements

- **Predictive Analytics**: ML models for spending prediction and anomaly detection
- **Tax Planning**: Integration with tax calculation engines and optimization recommendations
- **Multi-Currency Support**: Comprehensive handling of international currencies and exchange rates
- **Investment Integration**: Direct brokerage connections for portfolio management

5.4.3 Long-Term Vision

- **Financial Advisor Network**: Platform for connecting users with human financial advisors
- **Regulatory Compliance**: Formal compliance certifications for enterprise deployment
- **Embedded Finance**: APIs for embedding financial management in third-party applications
- **Decentralized Finance**: Integration with DeFi protocols and cryptocurrency management

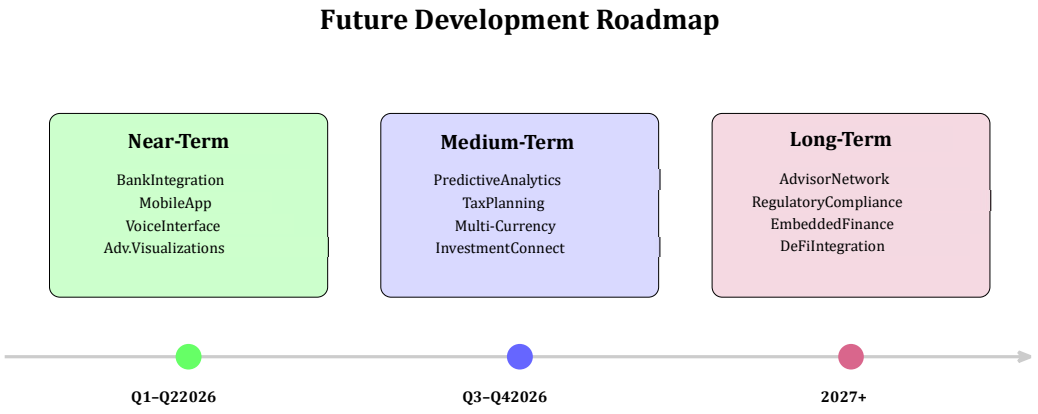


Figure 24. Future Development Roadmap

5.5 Closing Remarks

The Smart Personal Finance Management system represents a significant step forward in personal finance technology. By combining the operational depth of a real finance application with structured AI reasoning, the system provides users with a workspace where AI, data, collaboration, and automation all reinforce each other.

The multi-agent architecture demonstrates that complex financial reasoning can be decomposed into specialized tasks, each handled by a purpose-built agent, and then synthesized into coherent recommendations. The transparent workflow metadata builds trust by making the AI's reasoning process inspectable rather than opaque.

Most importantly, the project shows that AI in personal finance should not be a bolted-on chatbot but rather an integral part of a system that already understands users, organizations, financial data, and operational workflows. This integration creates a more complete and effective financial planning experience than either traditional finance applications or standalone AI tools can provide.

As AI capabilities continue to advance, the architecture and patterns established in this project provide a foundation for incorporating more sophisticated financial reasoning while maintaining the security, transparency, and human oversight that responsible financial software requires.

References

[1] Dogu Araci. Finbert: Financial sentiment analysis with pre-trained language models. *arXiv preprint arXiv:1908.10063*, 2019.

[2] Yi-Xiang Chen, Peng-Wei Zhang, and Zhi-Yuan Liu. Finbert: A pre-trained financial language representation model for financial text mining. *Proceedings of the International Joint Conference on Artificial Intelligence*, pages 4513–4519, 2023.

[3] LangChain. Langgraph: Building stateful multi-agent applications with llms. https://github.com/langchain-ai/langgraph, 2024. Accessed: 2024.

[4] Jason Lee, Sung-Min Park, and Hyun-Jung Kim. Personal finance management applications: A comprehensive review. *Journal of Financial Technology*, 8(2):145–167, 2020.

[5] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[7] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

[8] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[9] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Damodaran, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564*, 2023.